24CSEN1011

OBJECT ORIENTED PROGRAMMING

MODULE 5 EXCEPTION HANDLING AND CONCURRENCY

Exception Handling: Exception Hierarchy, Fundamentals, Consequences of an Uncaught Exception, Handling errors, Exception handling, Multiple Catch, Throwing and Rethrowing an Exception, Throwable, using finally, using throws, Creating Exception Subclasses.

Concurrency (Multithreading).

Inner classes and lambda expressions

EXCEPTION HANDLING

- An **exception** is an **unexpected event or error** that occurs during the execution of a program, which **disrupts the normal flow** of instructions.
- **Exception handling** is a **mechanism** that allows a program to **detect, handle, and recover** from runtime errors (exceptions) so that the normal flow of execution is maintained.
- **Adv**:It helps prevent the program from **abruptly terminating** or **crashing** when an error occurs.

Why Exceptions Occur

Exceptions may happen because of:

- Invalid user input
- Code mistakes
- Device or network failure
- Insufficient memory
- Dividing a number by zero
- Trying to open a file that doesn't exist

When these happen, Java provides a way to **handle** them — this is called **exception handling**.

Types of Errors

There are 4 main kinds of errors in Java:

1. **Compilation Error** – Mistakes in syntax (like missing ; or ）)
2. **Linker Error** – Missing class or library file or method not defined.
3. **Logical Error** – Program runs but gives wrong output(instead of sum, find difference)
4. **Runtime Error** – Error occurs while program is running(divide by zero)

1. Compilation Error

Occurs when there is a **syntax mistake** — the program cannot compile.

```
1  class CompileError {
2      public static void main(String[] args)
           {
3          int a = 5
4          System.out.println(a);
5      }
6  }
7
```

```
ERROR!
/tmp/flguqC8r2z/Main.java:3: error: ';'
    expected
        int a = 5
                ^
1 error
ERROR!
error: compilation failed

=== Code Exited With Errors ===
```

2. Linker Error

In Java, linker errors happen when a **method is called but not defined properly**, or the **class file is missing**.

```
1  class LinkerError {
2      public static void main(String[] args)
           {
3          display();  // method not defined
4      }
5  }
6
```

```
ERROR!
/tmp/U3YFSOqEr7/Main.java:3: error: cannot
    find symbol
        display();  // method not defined
        ^
  symbol:   method display()
  location: class LinkerError
1 error
ERROR!
error: compilation failed
```

3. Logical Error

The program compiles and runs, but gives the **wrong output** due to incorrect logic.

```java
class LogicalError {
    public static void main(String[] args) {
        int a = 10, b = 5;
        int sum = a - b;  // Logic mistake (should
            be a + b)
        System.out.println("Sum = " + sum);
    }
}
```

```
Sum = 5

=== Code Execution Successful ===
```

4. Runtime Error

Occurs **during execution** — for example, dividing by zero or invalid array access.(**ArithmeticException**)

```java
class RuntimeError {
    public static void main(String[] args) {
        int a = 10, b = 0;
        int c = a / b;  // Division by zero
        System.out.println(c);
    }
}
```

```
ERROR!
Exception in thread "main" java.lang.ArithmeticException
    : / by zero
    at RuntimeError.main(Main.java:4)

=== Code Exited With Errors ===
```

Runtime Errors Examples

1. **Input Errors** – Invalid data entered by user
2. **Device Errors** – Hardware or file not found
3. **Physical Limitation Errors** – Memory full or overflow

Merits of Exception Handling

- It allows the programmer to **fix errors** easily.
- It **prevents the program from stopping** suddenly.

1.Input Error – Invalid data entered by user:The program expects an integer, but the user entered a character — this causes an **Input Error**.

```java
import java.util.*;

class InputError {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();  // if user enters a letter (e.g. 'a'),
            error occurs
        System.out.println("Number = " + n);
    }
}
```

```
Enter a number: a
ERROR!
Exception in thread "main" java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:947)
    at java.base/java.util.Scanner.next(Scanner.java:1602)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2267)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2221)
    at InputError.main(Main.java:7)

=== Code Exited With Errors ===
```

2. FileNotFoundException:This error occurs when a **file or device** required by the program is **missing or inaccessible**.

```java
import java.io.File;
import java.io.FileReader;

public class DeviceErrorDemo {
    public static void main(String[] args) throws Exception {
        // Attempting to open a file that doesn't exist
        File file = new File("nonexistentfile.txt");
        FileReader fr = new FileReader(file); // This will throw
            FileNotFoundException
    }
}
```

```
ERROR!
Exception in thread "main" java.io.FileNotFoundException: nonexistentfile.txt (No such
    file or directory)
    at java.base/java.io.FileInputStream.open0(Native Method)
    at java.base/java.io.FileInputStream.open(FileInputStream.java:213)
    at java.base/java.io.FileInputStream.<init>(FileInputStream.java:152)
    at java.base/java.io.FileReader.<init>(FileReader.java:75)
    at DeviceErrorDemo.main(Main.java:8)

=== Code Exited With Errors ===
```

3.Physical Limitation Error – Memory full or overflow: The system doesn't have enough memory to create such a large array, so it throws **OutOfMemoryError** — a **Physical Limitation Error**.

```java
public class LargeArrayDemo {
    public static void main(String[] args) {
        // Attempting to create a huge array of 1 billion integers
        int[] arr = new int[1000000000]; // 1,000,000,000 elements

        System.out.println("Array created successfully!");
    }
}
```

```
ERROR!
Exception in thread "main" ERROR!
java.lang.OutOfMemoryError: Java heap space
    at LargeArrayDemo.main(Main.java:4)
    at java.base/java.lang.invoke.LambdaForm$DMH/0x00007e8f60030c00.invokeStatic
        (LambdaForm$DMH)
    at java.base/java.lang.invoke.LambdaForm$MH/0x00007e8f6012d400.invoke
        (LambdaForm$MH)
    at java.base/java.lang.invoke.Invokers$Holder.invokeExact_MT(Invokers$Holder)
    at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invokeImpl
        (DirectMethodHandleAccessor.java:154)

=== Code Exited With Errors ===
```

Maximum memory size for `int` in Java = 4 bytes = 32 bits = Maximum value 2,147,483,647(Two hundred fourteen crore seventy-four lakh eighty-three thousand six hundred forty-seven).

Exception Name	Explanation	Example
ArithmeticException	Division by zero	int a = 10 / 0;
ArrayIndexOutOfBoundsException	Accessing invalid array index	int arr[] = {1,2,3}; System.out.println(arr[5]);
NullPointerException	Using a null object reference	String s = null; System.out.println(s.length());
ClassCastException	Wrong object type casting	Object x = new Integer(5); String y = (String)x;
NegativeArraySizeException	Creating array with negative size	int arr[] = new int[-5];
NoClassDefFoundError	Missing class definition	Trying to use a class that is not found at runtime
NumberFormatException	Wrong input type (e.g., letter instead of number)	int n = Integer.parseInt("abc");

ArithmeticException

```java
class RuntimeError {
    public static void main(String[] args) {
        int a = 10, b = 0;
        int c = a / b;  // Division by zero
        System.out.println(c);
    }
}
```

```
ERROR!
Exception in thread "main" java.lang.ArithmeticException
    : / by zero
    at RuntimeError.main(Main.java:4)

=== Code Exited With Errors ===
```

ArrayIndexOutOfBoundsException

```java
class InvalidArrayAccess {
    public static void main(String[] args) {
        int numbers[] = {10, 20, 30};

        // Trying to access index 5 (which does not exist)
        System.out.println(numbers[5]);
    }
}
```

```
ERROR!
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
    Index 5 out of bounds for length 3
    at InvalidArrayAccess.main(Main.java:6)

=== Code Exited With Errors ===
```

NullPointerException

```java
public class NullPointerDemo {
    public static void main(String[] args) {
        String str = null; // str is null

        // This line will cause a NullPointerException
        System.out.println(str.length());
    }
}
```

```
ERROR!
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "String
    .length()" because "<local1>" is null
    at NullPointerDemo.main(Main.java:6)

=== Code Exited With Errors ===
```

NullPointerException:occurs

1.**Calling a method on a null object**

```
String str = null;
str.length();                                   // NPE occurs
```

2.**Accessing or modifying a field of a null object:**

```
class Person {
   String name;
}
Person p = null;
System.out.println(p.name);                     // NPE occurs
```

3.**Accessing elements in an array of objects that are null:**

```
String[] arr = new String[3];                   // all elements are null by default
System.out.println(arr[0].length());            // NPE occurs
```

4.**Passing null to a method that dereferences it:**

```
void printLength(String s) {
   System.out.println(s.length());
}
printLength(null);                              // NPE occurs
```

5.**Unboxing a null wrapper object to a primitive type:**

```
Integer num = null;
int n = num;                                    // NPE occurs during unboxing
```

ClassCastException occurs when the **actual object type** is not compatible with the **type you are trying to cast it to**.

```java
public class ClassCastDemo {
    public static void main(String[] args) {
        Object obj = new Integer(100); // obj actually holds an Integer

        // Attempting to cast Integer to String
        String str = (String) obj; // This will throw ClassCastException

        System.out.println(str);
    }
}
```

```
/tmp/d8wMHDlvTJ/Main.java:3: warning: [removal] Integer(int) in Integer has been
    deprecated and marked for removal
        Object obj = new Integer(100); // obj actually holds an Integer
                     ^
1 warning
ERROR!
Exception in thread "main" java.lang.ClassCastException: class java.lang.Integer
    cannot be cast to class java.lang.String (java.lang.Integer and java.lang.String
    are in module java.base of loader 'bootstrap')
    at ClassCastDemo.main(Main.java:6)

=== Code Exited With Errors ===
```

NegativeArraySizeException occurs when you try to create an array with a **negative size**. This is a **runtime exception**.

```java
public class NegativeArrayDemo {
    public static void main(String[] args) {
        int size = -5; // Negative size

        // Attempting to create an array with negative size
        int[] arr = new int[size]; // This will throw NegativeArraySizeException

        System.out.println("Array created successfully!");
    }
}
```

```
ERROR!
Exception in thread "main" java.lang.NegativeArraySizeException: -5
    at NegativeArrayDemo.main(Main.java:6)

=== Code Exited With Errors ===
```

NumberFormatException occurs when you try to **convert a string into a numeric type** (like `int, double,` etc.) and the string **does not have a valid format**. This is a **runtime exception**.

```java
public class NumberFormatDemo {
    public static void main(String[] args) {
        String str = "abc123"; // Not a valid number

        // Attempting to convert string to integer
        int num = Integer.parseInt(str); // This will throw
            NumberFormatException

        System.out.println("Number is: " + num);
    }
}
```

```
ERROR!
Exception in thread "main" java.lang.NumberFormatException: For input string: "abc123"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException
        .java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:662)
    at java.base/java.lang.Integer.parseInt(Integer.java:778)
    at NumberFormatDemo.main(Main.java:6)

=== Code Exited With Errors ===
```

Keywords to handle exception

Keyword	Definition	Syntax	Uses
try	Defines a block of code where exceptions might occur	`try { // code that may throw exception }`	To test code that may generate an exception
catch	Defines a block to handle the exception thrown in `try`	`catch(ExceptionType e) { // handle exception }`	To catch and handle exceptions so program continues
throw	Used to manually throw an exception	`throw new ExceptionType("message");`	To explicitly throw an exception in code
throws	Declares exceptions a method might throw	`void method() throws ExceptionType { // code }`	To inform the caller of the method about possible exceptions
finally	Defines a block that always executes after `try`/`catch`	`finally { // code to execute always }`	To release resources or execute cleanup code regardless of exceptions

With out Exception handling

```
class exc1 {
    public static void main(String args[]) {
        int a = 10, b = 0, c;
        System.out.println("THIS STATEMENT EXECUTED");
        c = a / b;  // This line will cause an exception
        System.out.println(c);
        System.out.println("THIS STATEMENT Never EXECUTED");
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac exc1.java

C:\Users\sylaj\OneDrive\oop program>java exc1
THIS STATEMENT EXECUTED
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at exc1.main(exc1.java:7)
```

Exception Handling

Exception Handling in Java is a mechanism to handle runtime errors, ensuring the normal flow of the program.
It prevents abnormal program termination by using keywords like `try`, `catch`, `throw`, `throws`, and `finally`.

Exception Hierarchy

All exceptions are part of the **`Throwable`** class hierarchy.

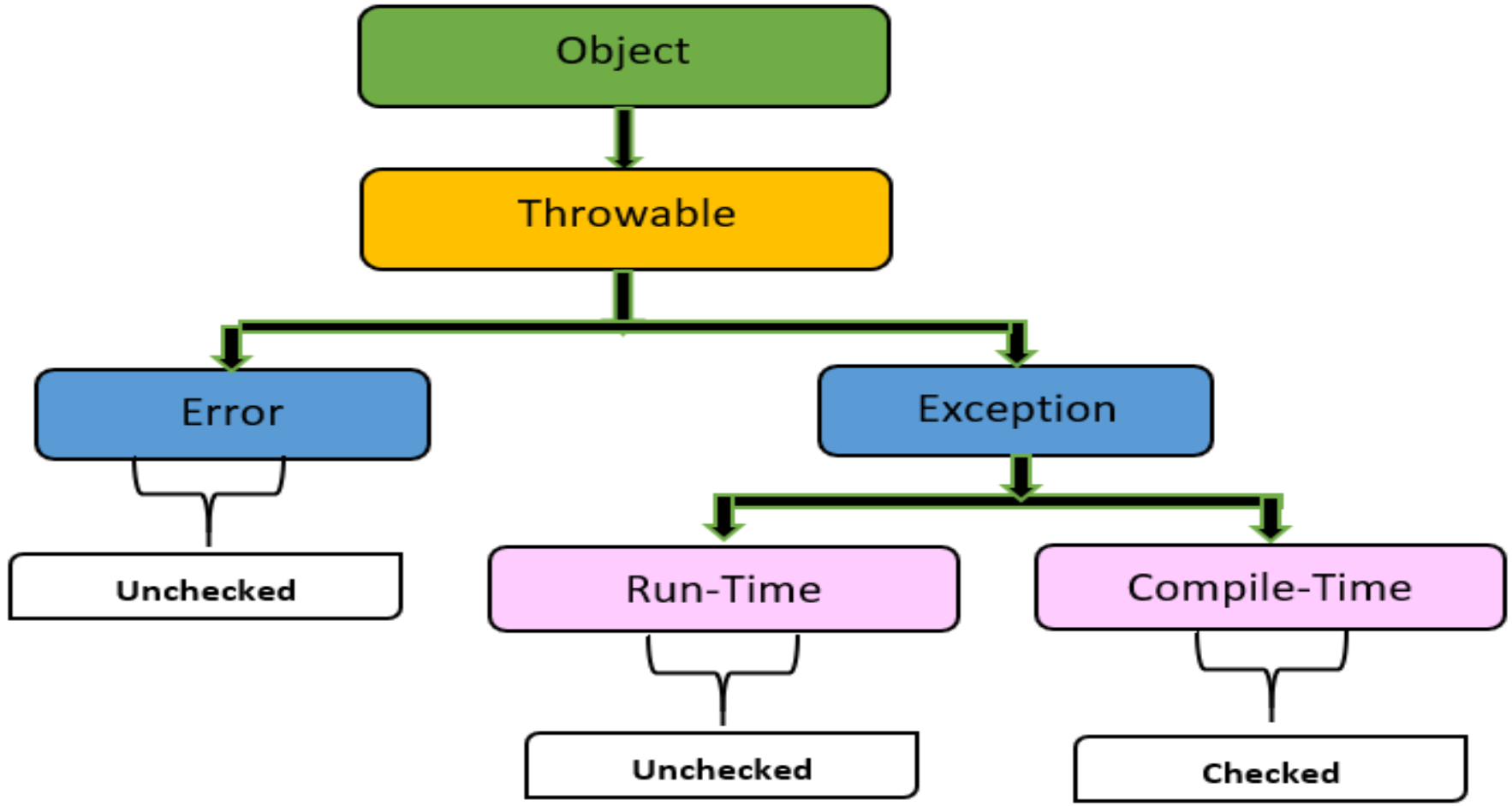
- **`Throwable`** is the superclass of all exceptions.
- It has two main subclasses:
  - **`Error`** → Serious problems that a program **cannot handle.**
  - **`Exception`** → Conditions a program might want to catch

Program to show Exception Hierarchy(Exception handling using try..catch)

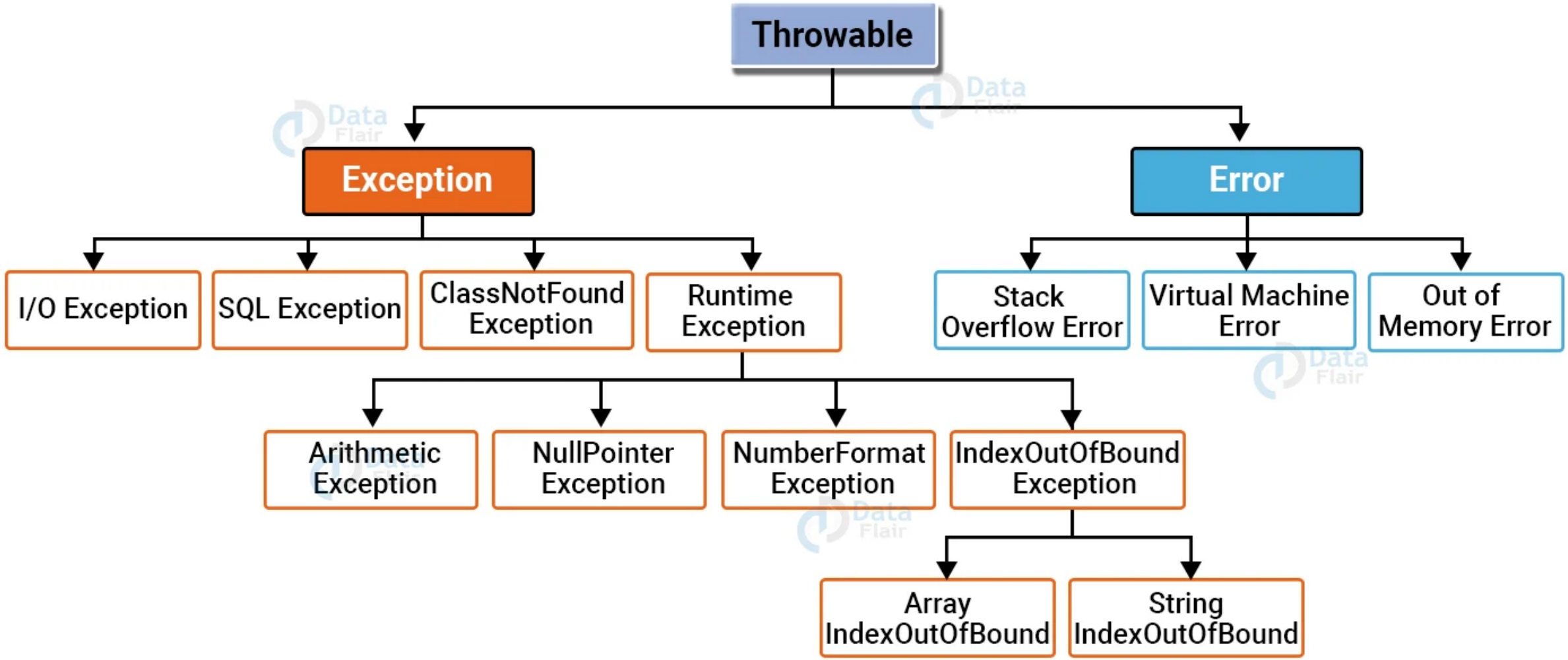
```
class ExceptionHierarchy {
    public static void main(String[] args) {
        try {
            int a = 10 / 0; // ArithmeticException occurs
        } catch (ArithmeticException e) {
            System.out.println("Caught an exception: " + e);
        }
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac ExceptionHierarchy.java

C:\Users\sylaj\OneDrive\oop program>java ExceptionHierarchy
Caught an exception: java.lang.ArithmeticException: / by zero
```


Object
Throwable
Error
Exception
Unchecked
Run-Time
Compile-Time
Unchecked
Checked

Hierarchy of Java Exceptions



Condition for try-catch

- **try block:** Write the code that **might cause an error**.
- **catch block:** Handles the error if it occurs, so the program **does not crash**.
- **Rule:** Every try must have **at least one catch**, otherwise it gives a **compilation error**.

When an exception happens:

- catch executes and prints the error message.
- Program continues running normally after the catch block.

Fundamentals of Exception Handling

- Exception handling uses **try-catch blocks** to detect and handle errors at runtime.
- `try` contains code that may throw an exception, and `catch` handles it.

Program to demonstrate basic exception handling

```
class BasicException {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;                    // exception
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero is not allowed");
        }
        System.out.println("Program continues normally...");
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac BasicException.java

C:\Users\sylaj\OneDrive\oop program>java BasicException
Error: Division by zero is not allowed
Program continues normally...
```

Consequences of an Uncaught Exception

If an exception is not handled, the program terminates **abnormally** and displays an **error message.**

Example:Program to show consequence of an uncaught exception

```
class UncaughtExample {
    public static void main(String[] args) {
        int a = 10 / 0; // no try-catch, so program stops here
        System.out.println("This line will not execute");
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac UncaughtExample.java

C:\Users\sylaj\OneDrive\oop program>java UncaughtExample
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at UncaughtExample.main(UncaughtExample.java:3)
```

Handling Errors

- Errors represent serious issues that cannot be handled (like memory overflow).
- But **minor runtime errors** (like invalid input) **can be handled** using try-catch.

Example: Program to handle runtime errors

```
import java.util.*;
class HandleError {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        try {
            int num = sc.nextInt();
            System.out.println("You entered: " + num);
        } catch (InputMismatchException e) {
            System.out.println("Error: Please enter a valid integer");
        }
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac HandleError.java

C:\Users\sylaj\OneDrive\oop program>java HandleError
Enter a number: a
Error: Please enter a valid integer

C:\Users\sylaj\OneDrive\oop program>javac HandleError.java

C:\Users\sylaj\OneDrive\oop program>java HandleError
Enter a number: 10
You entered: 10
```

Note:Giving character as input instead of integer(InputMismatchException)

With Exception Handling (using try .... Catch)

```
class exc2 {
    public static void main(String args[]) {
        int a = 10, b = 0, c; // Declare and initialize variables

        System.out.println("THIS STATEMENT EXECUTED");
    // Prints before the exception occurs

        try {
            c = a / b;
            // This will cause ArithmeticException (division by zero)

            System.out.println(c);
            // This line will NOT execute because exception occurs above

            System.out.println("THIS STATEMENT Never EXECUTED");
            // This line also will NOT execute
        }
        catch (ArithmeticException e) {
            System.out.println("i am in error");
            // Catches the ArithmeticException and prints the error message
        }
        System.out.println("Now i am working hereafter...");
        // Program continues normally after handling exception
    }
}
```

Output:

```
C:\Users\sylaj\OneDrive\oop program>javac  exc2.java

C:\Users\sylaj\OneDrive\oop program>java  exc2
THIS STATEMENT EXECUTED
i am in error
Now i am working hereafter...
```

Catch block with some additional code: (In catch block you may give the second chance to the user to correct their errors:)

```
import java.util.*; // Importing Scanner class for user input

class exc3
{
   public static void main(String args[])
   {
      int a, b, c;
      Scanner sc = new Scanner(System.in); // Scanner object for input

        try
      {
      // Asking user to enter two numbers
         System.out.println("Enter two numbers:");
         a = sc.nextInt();
         b = sc.nextInt();

         // Performing division operation
          c = a / b;  // May cause ArithmeticException if b = 0
         System.out.println("Result: " + c);
      }
      catch (ArithmeticException e)
      {
         // This block executes when division by zero occurs
         System.out.println("Don't give zero for the second number!");

         // Giving the user another chance to enter correct values
         System.out.println("Enter two valid numbers again:");
         a = sc.nextInt();
         b = sc.nextInt();

         // Performing division again with new input
         c = a / b;
         System.out.println("Corrected Result: " + c);
      }

      // Program continues execution even after exception handling
      System.out.println("Now I am working hereafter...");
   }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac exc3.java

C:\Users\sylaj\OneDrive\oop program>java exc3
Enter two numbers:
10 0
Don't give zero for the second number!
Enter two valid numbers again:
10 5
Corrected Result: 2
Now I am working hereafter...

C:\Users\sylaj\OneDrive\oop program>javac exc3.java

C:\Users\sylaj\OneDrive\oop program>java exc3
Enter two numbers:
16 4
Result: 4
Now I am working hereafter...
```

Try with multiple catches: Multiple catch blocks handle different types of exceptions from a single try block.
Each catch block handles a specific type of exception.

```
import java.util.*;
class exc4
{
   public static void main(String args[])
   {
      int a, b, c,d;
      int d[] = new int[3];   // array of size 3
      Scanner scr = new Scanner(System.in);


       try
      {
        a = scr.nextInt();
        b = scr.nextInt();
        c = a / b;       // may cause ArithmeticException
        System.out.println(c);
       d[5] = 10;   // may causerrayIndexOutOfBoundsException
        System.out.println(d[2]);
      }
      catch (ArithmeticException e)
      {
        System.out.println("don't give zero for the second number");
      }
      catch (ArrayIndexOutOfBoundsException e)
      {
        System.out.println("don't go beyond the array limit");
      }

      System.out.println("Now I am working hereafter...");
 // executes after exception handling
   }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac exc4.java

C:\Users\sylaj\OneDrive\oop program>java exc4
10 0
don't give zero for the second number
Now I am working hereafter...

C:\Users\sylaj\OneDrive\oop program>javac exc4.java

C:\Users\sylaj\OneDrive\oop program>java exc4
10 5
2
don't go beyond the array limit
Now I am working hereafter...
```

Throwing an Exception

We use the `throw` keyword to manually throw an exception when a specific condition occurs.

Example: Program to demonstrate throwing an exception using 'throw'

```
class ThrowExample {
    public static void main(String[] args) {
        int age = 15;

        // If the condition is true, manually throw an exception
        if (age < 18) {
            throw new ArithmeticException("Not eligible to vote");
        }

        System.out.println("Eligible to vote");
    }
}
```

Output:

```
C:\Users\sylaj\OneDrive\oop program>javac ThrowExample.java

C:\Users\sylaj\OneDrive\oop program>java ThrowExample
Exception in thread "main" java.lang.ArithmeticException: Not eligible to vote
        at ThrowExample.main(ThrowExample.java:7)
```

'throw' keyword

'throw' is used to **manually** create and throw an exception

```
import java.io.*;
class thro1_1
{
   public static void main(String arg[])
   {
      // Manually throwing an ArithmeticException
      throw new ArithmeticException();   // it doesn't show the message '/ by zero'
                        (or)
      // throw new NullPointerException();   // can also be used to throw other exceptions
   }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac thro1_1.java

C:\Users\sylaj\OneDrive\oop program>java thro1_1
Exception in thread "main" java.lang.ArithmeticException
        at thro1_1.main(thro1_1.java:7)
```

Program to demonstrate user-defined exception using 'throw'

```
import java.io.*;
import java.util.*;

class thro1
{
   public static void main(String arg[])
   {
      int a = 100;
      int b;
      Scanner sc = new Scanner(System.in);

      System.out.println("Enter a number:");
      b = sc.nextInt();

      // We may call the exception for our own conditions using 'throw'
      if (b == 5)
      {
         throw new ArithmeticException("The number should not be 5");
      }

      System.out.println("Thank you");
   }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac thro1.java

C:\Users\sylaj\OneDrive\oop program>java thro1
Enter a number:
10
Thank you

C:\Users\sylaj\OneDrive\oop program>javac thro1.java

C:\Users\sylaj\OneDrive\oop program>java thro1
Enter a number:
5
Exception in thread "main" java.lang.ArithmeticException: The number should not be 5
        at thro1.main(thro1.java:19)
```

Program to demonstrate **unreachable statement** after using **`throw`** in Java

After 'throw', no statement can be executed because it terminates the flow (unreachable statement)

```
class thro2
{
   public static void main(String arg[])
   {
      // After 'throw', no statement can be executed because it terminates the program (unreachable statement)
      throw new ArithmeticException("please give multiples of 500");

      System.out.println("I am not executed");  // unreachable statement
   }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac thro2.java
thro2.java:8: error: unreachable statement
        System.out.println("I am not executed");  // unreachable statement
        ^
1 error
```

Rethrowing an Exception

When an exception is caught in one `try-catch` block and then thrown again using `throw`, it's called rethrowing.

Example: Program to demonstrate rethrowing same exception

```
class RethrowExample {
    public static void main(String[] args) {
        try {
            try {
                int a = 5 / 0;              // generates ArithmeticException
            } catch (ArithmeticException e) {
                System.out.println("Exception caught: " + e);
                throw e;                    // rethrowing the exception
            }
        } catch (ArithmeticException e) {
            System.out.println("Exception handled again: " + e);
        }
    }
}
```

Output:

```
C:\Users\sylaj\OneDrive\oop program>javac RethrowExample.java

C:\Users\sylaj\OneDrive\oop program>java RethrowExample
Exception caught: java.lang.ArithmeticException: / by zero
Exception handled again: java.lang.ArithmeticException: / by zero
```

Rethrowing a New Exception

```
class RethrowInMain {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;    // This causes ArithmeticException
        }
        catch (ArithmeticException e) {
            System.out.println("Caught: " + e.getMessage());
            // Rethrowing a new exception
            throw new RuntimeException("Rethrowing as RuntimeException", e);
        }
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac RethrowInMain.java

C:\Users\sylaj\OneDrive\oop program>java RethrowInMain
Caught: / by zero
Exception in thread "main" java.lang.RuntimeException: Rethrowing as RuntimeException
        at RethrowInMain.main(RethrowInMain.java:9)
Caused by: java.lang.ArithmeticException: / by zero
        at RethrowInMain.main(RethrowInMain.java:4)
```

Throwable Class

Throwable is the superclass of all errors and exceptions in Java. It has two main subclasses — **Error** and **Exception.**

Example: Program showing **Throwable** class methods

```
class ThrowableExample {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;
        } catch (ArithmeticException e) {
            // Throwable methods to get details about the exception
            System.out.println("Message: " + e.getMessage());
            System.out.println("String form: " + e.toString());
            e.printStackTrace();                    // prints full stack trace
        }
    }
}
```

Output:

```
C:\Users\sylaj\OneDrive\oop program>javac ThrowableExample.java

C:\Users\sylaj\OneDrive\oop program>java ThrowableExample
Message: / by zero
String form: java.lang.ArithmeticException: / by zero
java.lang.ArithmeticException: / by zero
        at ThrowableExample.main(ThrowableExample.java:4)
```

Using finally

`finally` block will **always execute code** inside it (like **closing** files or connections) whether an exception occurs or not.

Example: Program to demonstrate 'finally' block execution

```
class FinallyExample {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;                 // may cause exception
        } catch (ArithmeticException e) {
            System.out.println("Exception caught");
        } finally {
            // This block always executes whether exception occurs or not
            System.out.println("Finally block executed");
        }
    }
}
```

Output:

```
C:\Users\sylaj\OneDrive\oop program>javac FinallyExample.java

C:\Users\sylaj\OneDrive\oop program>java FinallyExample
Exception caught
Finally block executed
```

finally block

(The finally block always executes whether an exception occurs or not)

```
import java.util.*;
class exc6
{
   public static void main(String args[])
   {
      int a, b, c;
      int d[] = new int[3];      // array of size 3
      Scanner scr = new Scanner(System.in);

      try
      {
         a = scr.nextInt();
         b = scr.nextInt();
         c = a / b;            // may cause ArithmeticException
         System.out.println(c);
         d[1] = 100;           // valid array index
      }
      catch (ArithmeticException e)
      {
         System.out.println("error in program");
      }
      finally
      {
         System.out.println("Thank you");  // always executed
      }

      System.out.println("Now I am working hereafter...");
// executes after finally
   }
}
```

```
C:\Users\sylaj\OneDrive\oop program>java exc6
10 0
error in program
Thank you
Now I am working hereafter...

C:\Users\sylaj\OneDrive\oop program>javac exc6.java

C:\Users\sylaj\OneDrive\oop program>java exc6
10 5
2
Thank you
Now I am working hereafter...
```

'throws' keyword(In 'throws', the system handles the exception instead of the user)

```
import java.util.*;
class throws1
{
   public static void main(String args[]) throws ArithmeticException
   {
      int a, b, c;
      Scanner scr = new Scanner(System.in);

      a = scr.nextInt();
      b = scr.nextInt();
      c = a / b;              // may cause ArithmeticException
      System.out.println(c);     // executed only if no exception occurs
   }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac throws1.java

C:\Users\sylaj\OneDrive\oop program>java throws1
10 0
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at throws1.main(throws1.java:14)

C:\Users\sylaj\OneDrive\oop program>javac throws1.java

C:\Users\sylaj\OneDrive\oop program>java throws1
10 5
2
```

Example:throws

```
import java.io.*;

class ThrowsExample {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
       br.close();                          // ❌ Closing the stream before reading
        System.out.print("Enter your name: ");
        String name = br.readLine();   // may cause IOException

        System.out.println("Hello, " + name);
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac Throws1Example.java

C:\Users\sylaj\OneDrive\oop program>java Throws1Example
Enter your name: Ramu
Hello, Ramu
```

```
C:\Users\sylaj\OneDrive\oop program>javac Throws1Example.java

C:\Users\sylaj\OneDrive\oop program>java Throws1Example
Enter your name: Exception in thread "main" java.io.IOException: Stream closed
        at java.base/java.io.BufferedReader.ensureOpen(BufferedReader.java:121)
        at java.base/java.io.BufferedReader.implReadLine(BufferedReader.java:362)
        at java.base/java.io.BufferedReader.readLine(BufferedReader.java:347)
        at java.base/java.io.BufferedReader.readLine(BufferedReader.java:436)
        at Throws1Example.main(Throws1Example.java:8)
```

Program to catch all kinds of exceptions using the Exception class

```
import java.util.*;
class exc5
{
   public static void main(String args[])
   {
      int a, b, c;
      int d[] = new int[3];       // array of size 3
      Scanner sc = new Scanner(System.in);

      try
      {
         a = sc.nextInt();
         b = sc.nextInt();
         c = a / b;               // may cause ArithmeticException
         System.out.println(c);
         d[7] = 10;               // may cause ArrayIndexOutOfBoundsException
         System.out.println(d[2]);
      }
      catch (Exception e)         // catches all types of exceptions
      {
         System.out.println("error in program");
      }

      System.out.println("Now I am working hereafter...");
// executed after handling exception
   }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac exc5.java

C:\Users\sylaj\OneDrive\oop program>java exc5
10 0
error in program
Now I am working hereafter...

C:\Users\sylaj\OneDrive\oop program>javac exc5.java

C:\Users\sylaj\OneDrive\oop program>java exc5
10 5
2
error in program
Now I am working hereafter...
```

Creating Exception Subclasses

Sometimes, Java's built-in exceptions(**user-defined or custom exception**) are not enough.

So, we can **create our own exception class** to handle special errors.

We do this by **extending** either:

Exception → for **checked exceptions**, or

RuntimeException → for **unchecked exceptions**.

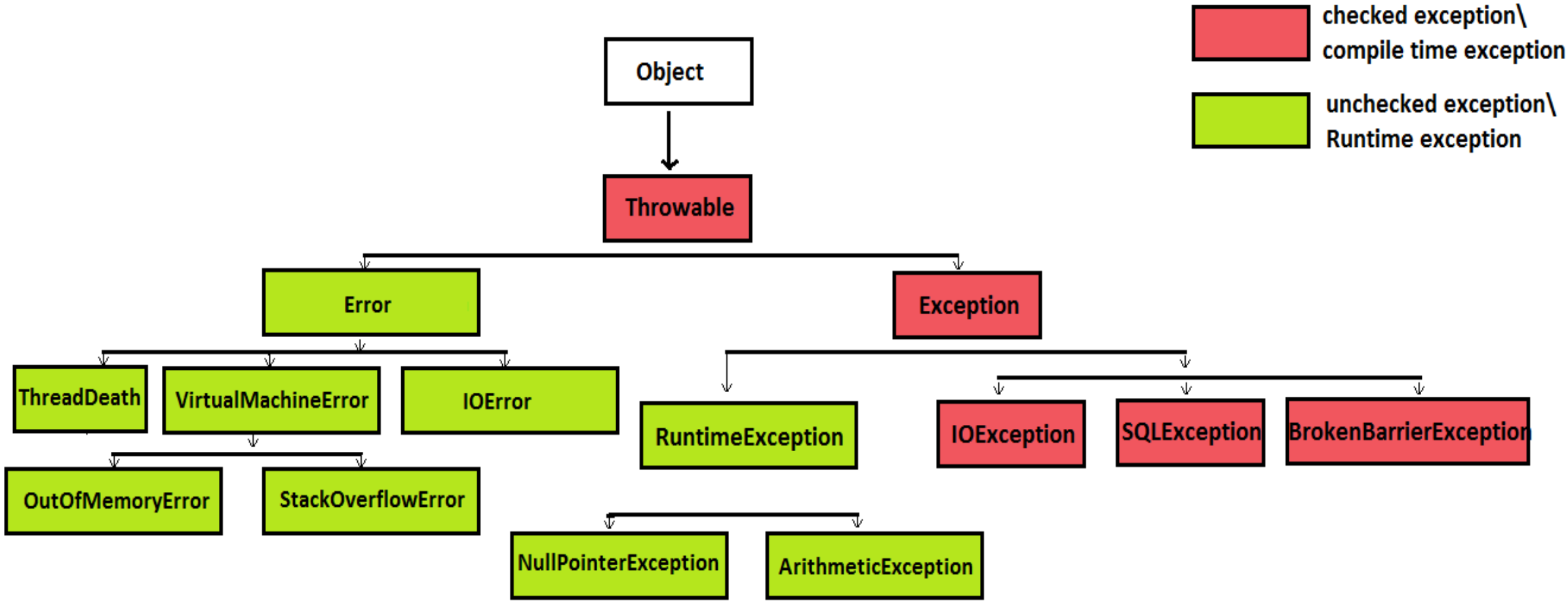
Note:

extends **Exception** → checked exception (**must use try-catch or throws**).

extends **RuntimeException** → unchecked exception (**optional handling**).

Always call **super(message)** to pass a custom **error message**.

Condition	Checked Exception	Unchecked Exception
When it occurs	Found at **compile time**	Found at **runtime**
Compiler checking	Compiler **forces** you to handle it	Compiler **does not check**
Handling	Must use `try-catch` or declare using `throws`	Handling is **optional**
Base class	Inherits from **Exception**	Inherits from **RuntimeException**
Example exceptions	`IOException`, `SQLException`, `FileNotFoundException`	`NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`
Program behavior	If not handled → **compile-time error**	If not handled → program **crashes** at runtime
Custom creation	Extend **Exception** class	Extend **RuntimeException** class


checked exception\
compile time exception
unchecked exception\
Runtime exception
Object
Throwable
Error
Exception
ThreadDeath
VirtualMachineError
IOError
RuntimeException
IOException
SQLException
BrokenBarrierException
OutOfMemoryError
StackOverflowError
NullPointerException
ArithmeticException

Syntax for checked exception

```
class MyException extends Exception {                    // checked exception
    public MyException(String message) {
        super(message);                                  // call the parent Exception class constructor
    }
}
```

Syntax for unchecked exception

```
class MyException extends RuntimeException {             // unchecked exception
    public MyException(String message) {
        super(message);                                  // call the parent Exception class constructor
    }
}
```

Using extends Exception (Checked Exception)

```
// Custom checked exception class
class MyException extends Exception {
    // Constructor that takes a message
    MyException(String msg) {
        super(msg);  // Call parent class (Exception) constructor
    }
}

public class Example1 {
    public static void main(String[] args) {
        try {
        // Manually throw the custom exception
            throw new MyException("This is a checked exception");
        }
        // Catch block to handle the exception
        catch (MyException e) {
            System.out.println("Caught: " + e.getMessage());
    }
 }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac Example1.java

C:\Users\sylaj\OneDrive\oop program>java Example1
Caught: This is a checked exception
```

Using extends RuntimeException (Unchecked Exception)

```
// Custom unchecked exception class

class MyRuntimeException extends RuntimeException {
    // Constructor with custom message
    MyRuntimeException(String msg) {
        super(msg);  // Call parent class (RuntimeException) constructor
    }
}
public class Example2 {
    public static void main(String[] args) {
        // Throwing exception directly (no need for try–catch)
        throw new MyRuntimeException("This is an unchecked exception");

        // The next line will not run because the program stops when exception occurs
        // System.out.println("End of program");
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac Example2.java

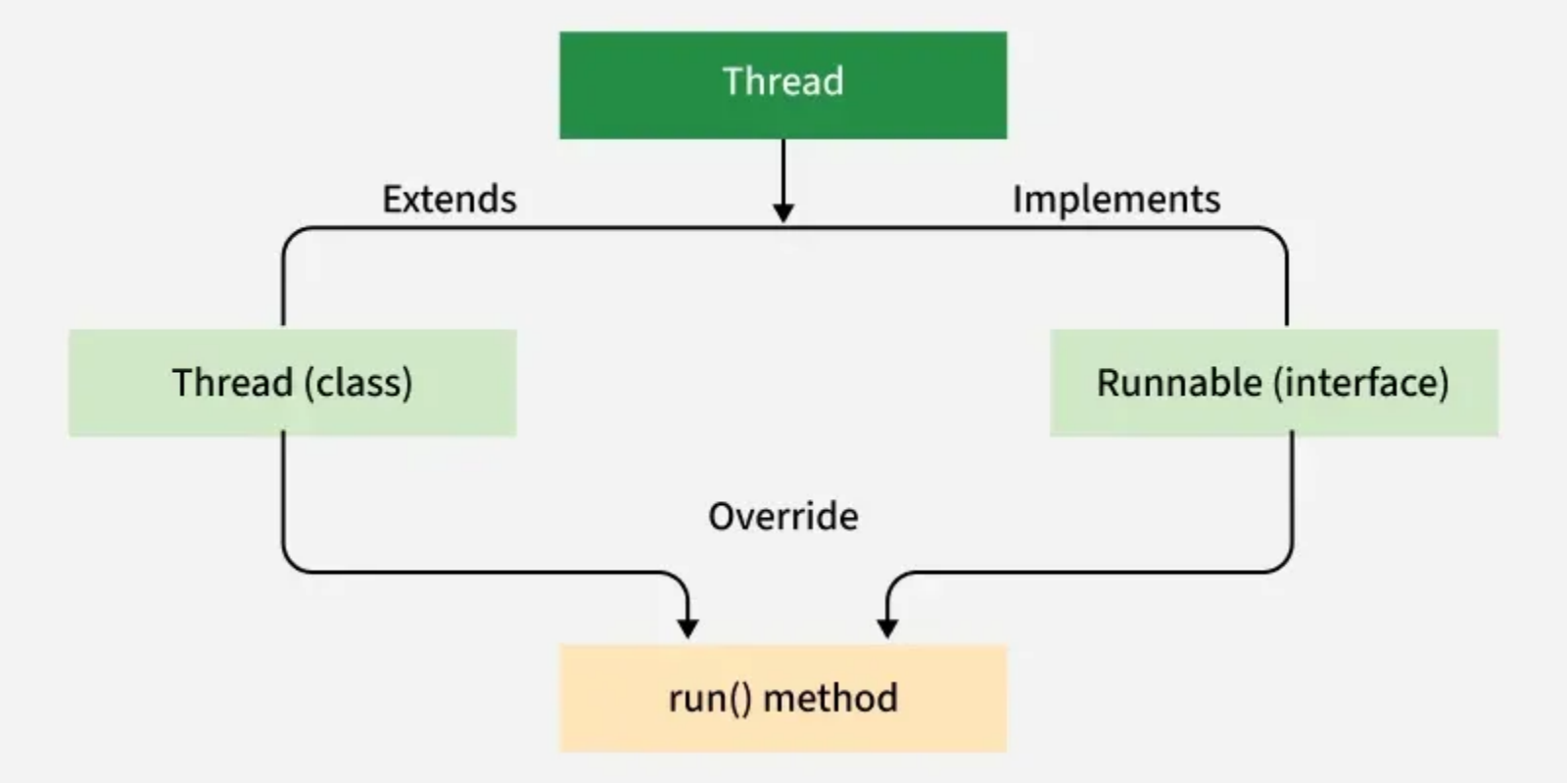
C:\Users\sylaj\OneDrive\oop program>java Example2
Exception in thread "main" MyRuntimeException: This is an unchecked exception
        at Example2.main(Example2.java:10)
```

Threads

A Java thread is the **smallest unit** of execution within a program. It is a lightweight subprocess that runs independently but shares the same memory space as the process, allowing multiple tasks to execute concurrently.

Create Threads in Java

We can create threads in java using two ways



Using Thread Class(create a new class that **extends** the Thread class and **override** its run() method.)

Syntax

```
class ClassName extends Thread {
    public void run() {
        // code to be executed by the thread
    }

    public static void main(String[] args) {
        ClassName t = new ClassName();  // create thread  object
        t.start();                      // start the thread (calls run())
    }
}
```

Using Runnable Interface(create a class that **implements the Runnable interface** and write your logic inside the run() method.
Then, pass this object to a **Thread** object.)

Syntax

```
class ClassName implements Runnable {
    public void run() {
     // code to be executed by the thread
    }

    public static void main(String[] args) {
// Create a Thread object and pass an instance of ClassName
        classname obj = new classname();  // create runnable object
       Thread t = new Thread(obj);        // pass to Thread
       t.start();                         // start the thread
    }
}
```

Example 1: Extending Thread Class

```
import java.util.*;
// Create a class that extends Thread
class MyThread extends Thread {
    // Override the run() method
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Thread is running... " + i);
        }
    }
}

// Main class
public class ThreadExample {
    public static void main(String[] args) {
        MyThread t = new MyThread();       // Create thread object  --> NEW state
        t.start();                         // Start the thread --> RUNNABLE → RUNNING

        System.out.println("Main thread is running...");
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac ThreadExample.java

C:\Users\sylaj\OneDrive\oop program>java ThreadExample
Thread running: 1
Thread running: 2
Thread running: 3
```

Example 2: Implementing Runnable Interface

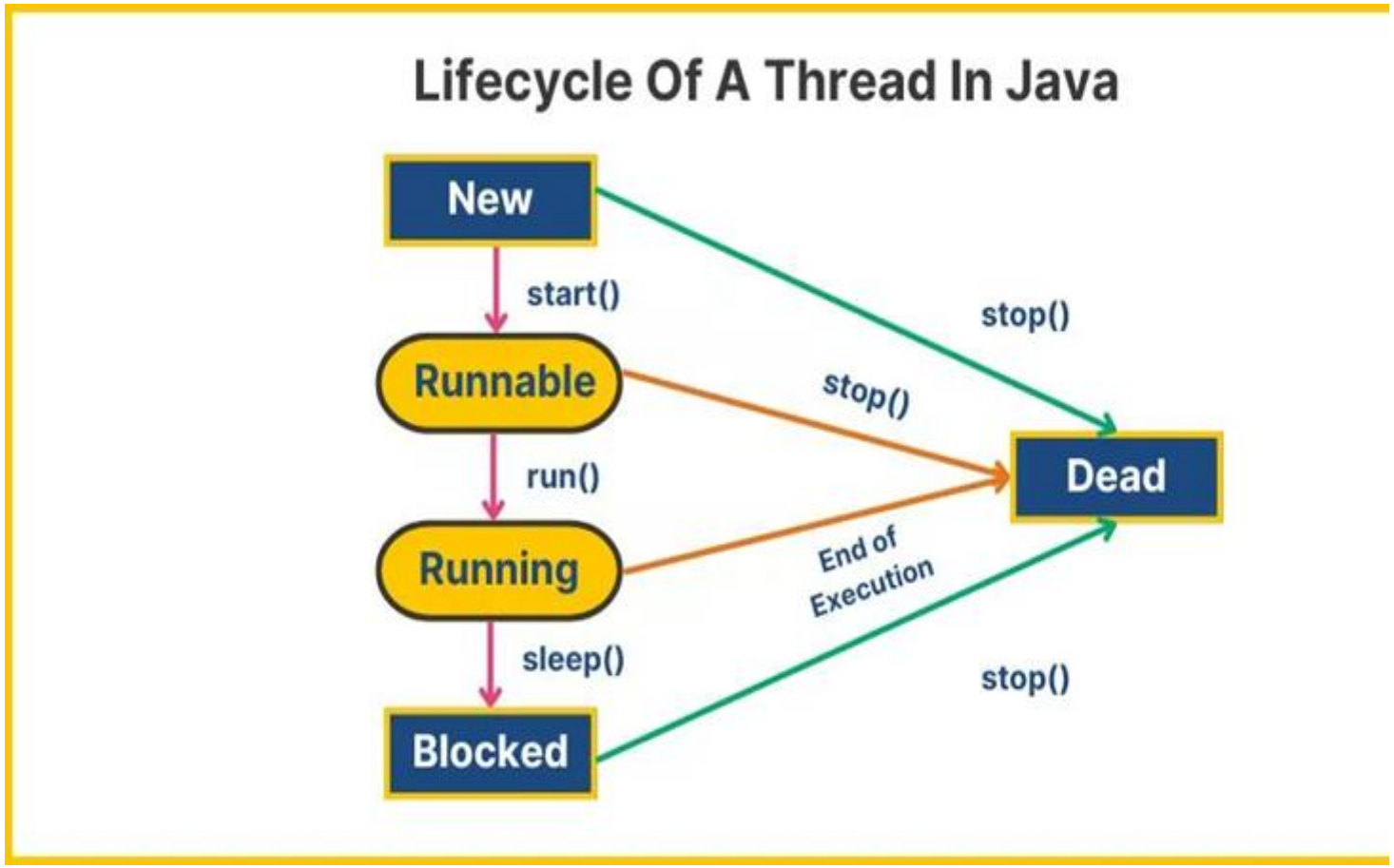
```
class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Runnable thread: " + i);
        }
    }
}
// Main class
public class RunnableExample2 {
    public static void main(String[] args) {
        MyRunnable obj = new MyRunnable();          // create Runnable object
        Thread t = new Thread(obj);                 // pass Runnable object to Thread
        t.start();                                  // start the thread

        System.out.println("Main thread is running...");
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac RunnableExample2.java

C:\Users\sylaj\OneDrive\oop program>java RunnableExample2
Main thread is running...
Runnable thread: 1
Runnable thread: 2
Runnable thread: 3
```

Life Cycle of a Thread



A thread in Java goes through **five main states** during its life cycle.

1. New State

When a thread is created but **not yet started**.

Created using the new keyword.

```
Thread t = new Thread();  // Thread is in NEW state
```

2. Runnable State

When the start() method is called.

The thread is **ready to run** and waiting for CPU time.

```
t.start();  // Thread enters RUNNABLE state
```

3. Running State

When the thread **gets CPU** and actually starts executing the run() method.

```
public void run() {
   System.out.println("Thread is running...");
}
```

4. Blocked / Waiting / Sleeping State

When a current thread is **temporarily inactive**.

```
Thread.sleep(1000);  // Thread goes to TIMED_WAITING state
```

5. Terminated (Dead) State

After the run() method completes execution, the thread is **terminated**.

```
System.out.println("Thread finished.");  // Thread is DEAD
```

Concurrency (Multithreading)

- **Concurrency (Multithreading)** means executing **multiple threads simultaneously**.
- A **thread** is a small unit of a process that runs independently.
- It helps to perform multiple tasks at the same time — like downloading a file while playing music.

Syntax:

Two ways to create a thread:

1. Extending the `Thread` class
2. Implementing the `Runnable` interface

CREATE A THREAD USING THREAD CLASS USING EXTENDS KEYWORD (Multithreading)

```
class One extends Thread {  // Create a class that extends Thread
  // run() method is called when the thread starts
    public void run() {
        for (int n = 0; n < 20; n++) {
            System.out.println("I am in child thread: " + n);
        }
System.out.println("Exit child thread");
    }
}

public class Thread6 {
    public static void main(String[] args) throws
InterruptedException {

        One o1 = new One();   // Create object of child thread class

        o1.start();          // run() will be in the quueue
        Thread.sleep(10);     // Pause main thread for 10 milliseconds

        System.out.println("Exit main thread");
    }
}
```

Note:

➤ The class extends the Thread class.

➤ It must override the run() method (this is the entry point for the new thread).

➤ The start() method is called to begin execution of the new thread.

Note: If the main thread finishes before the child thread has completed,

then the Java runtime system may terminate before the child thread finishes.

*/

```
C:\Users\sylaj\OneDrive\oop program>javac Thread6.java

C:\Users\sylaj\OneDrive\oop program>java Thread6
I am in main thread
I am in child thread: 0
I am in child thread: 1
I am in child thread: 2
I am in child thread: 3
I am in child thread: 4
I am in child thread: 5
I am in child thread: 6
I am in child thread: 7
I am in child thread: 8
I am in child thread: 9
I am in child thread: 10
I am in child thread: 11
I am in child thread: 12
I am in child thread: 13
I am in child thread: 14
Exit main thread
I am in child thread: 15
I am in child thread: 16
I am in child thread: 17
I am in child thread: 18
I am in child thread: 19
Exit child thread
```

```
C:\Users\sylaj\OneDrive\oop program>java Thread6
I am in main thread
I am in child thread: 0
I am in child thread: 1
I am in child thread: 2
I am in child thread: 3
I am in child thread: 4
I am in child thread: 5
I am in child thread: 6
I am in child thread: 7
I am in child thread: 8
I am in child thread: 9
Exit main thread
I am in child thread: 10
I am in child thread: 11
I am in child thread: 12
I am in child thread: 13
I am in child thread: 14
I am in child thread: 15
I am in child thread: 16
I am in child thread: 17
I am in child thread: 18
I am in child thread: 19
Exit child thread
```

Inner Classes

•An **inner class** is a **class declared inside another class**.

It helps **group related classes together**, makes the code **more readable**, and allows the inner class to **access private members** of the outer class.

Syntax:

```
class OuterClass {
    // data members of outer class
    int x = 10;   //private
    // inner class
    class InnerClass {
        void display() {
            System.out.println("Value of x = " + x);
        }
    }
}
```

Note:

Inner class **can access** variables and methods of the **outer class** (even if private).

To create an inner class object →**Outer.Inner obj = outerObj.new Inner();**

It helps in **encapsulation** and **logical grouping** of classes.

Example of Inner Class

```
class Outer {
    int num = 100;  // data member of outer class

    // inner class
    class Inner {
        void show() {
            System.out.println("Value of num = " + num);
        }
    }
}

public class InnerClassExample2 {
    public static void main(String[] args) {
        // Create object of outer class
        Outer outer = new Outer();

        // Create object of inner class using outer object
        Outer.Inner inner = outer.new Inner();

        // Call method of inner class
        inner.show();
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac InnerClassExample2.java

C:\Users\sylaj\OneDrive\oop program>java InnerClassExample2
Value of num = 100
```

Example Program:static inner class (Type of inner class)

```
class Outer {
    static int x = 10;

    static class Inner {
        void show() {
            System.out.println("Value of x = " + x);
        }
    }
}

public class InnerClassExample1 {
    public static void main(String[] args) {
        Outer.Inner inner = new Outer.Inner();  // directly using Outer.Inner
        inner.show();    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac InnerClassExample1.java

C:\Users\sylaj\OneDrive\oop program>java InnerClassExample1
Value of x = 10
```

Lambda Expressions

- A **Lambda Expression** is a **short form of writing** anonymous functions (functions without a name). It is mainly used to write **short, one-time-use methods** — especially when working with **threads, event handling, or functional interfaces**.

- Introduced in **Java 8** to simplify code, especially with functional interfaces.

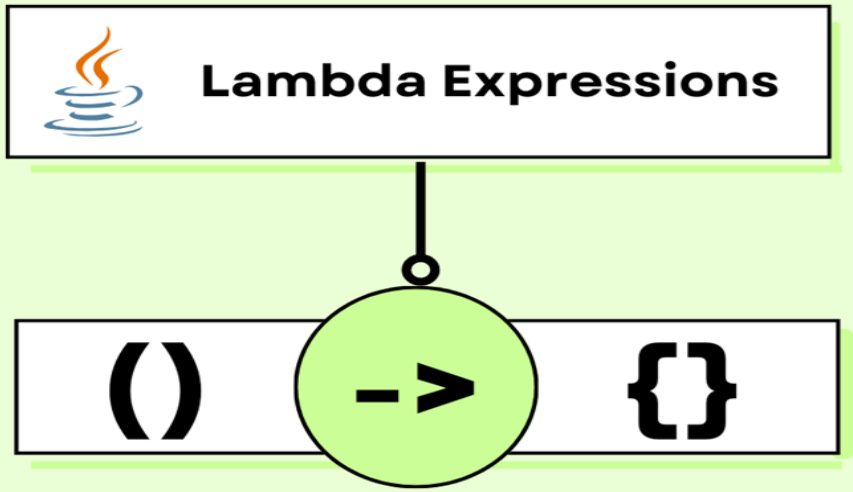
- **Syntax (Java Lambda Expression)**

(parameter_list) -> { body }

parameter_list : **variables** used in the function.

-> :arrow token, called **lambda operator**.

{ body } :**code** to execute.



Lambda Syntax

- No arguments:
```
() -> System.out.println("Hello")
```
- One argument:
```
s -> System.out.println(s)
```
- Two arguments:
```
(x, y) -> x + y
```
- With explicit argument types:
```
(Integer x, Integer y) -> x + y
```
- Multiple statements:
```
(x, y) -> {
    System.out.println(x);
    System.out.println(y);
    return (x + y);
};
```

Example:Lambda expression with no parameter,single parameter,two parameter

```
import java.util.*;

interface MyLambda {
    void display();
}

interface MyLambda1 {
    void show(String name);
}

interface MyLambda2 {
    int add(int a, int b);
}

public class LambdaExamples {
    public static void main(String[] args) {
        // No parameter
        MyLambda l1 = () -> System.out.println("Hello”);
        l1.display();

        // Single parameter
        MyLambda1 l2 = (name) -> System.out.println("Hello, " + name + "!");
        l2.show("Akshikha");

        // Two parameters
        MyLambda2 l3 = (a, b) -> a + b;
        System.out.println("Sum: " + l3.add(10, 20));
    }
}
```

Output:

```
C:\Users\sylaj\OneDrive\oop program>javac LambdaExamples.java

C:\Users\sylaj\OneDrive\oop program>java LambdaExamples
Hello
Hello, Akshikha!
Sum: 30
```

Lambda Expression with Explicit Arguments & Multiple Statements

```
import java.util.*;

// Functional interface with a method to add two numbers
interface Calculator {
    int add(int a, int b);
}

public class LambdaExplicitMultiStatement {
    public static void main(String[] args) {

        // Lambda expression with explicit arguments and multiple statements
        Calculator calc = (int a, int b) -> {
            System.out.println("Adding numbers: " + a + " and " + b);          // statement 1
            int sum = a + b;                                                   // statement 2
            return sum;                                                        // statement 3 (must return)
        };

        // Call the lambda expression
        int result = calc.add(10, 20);
        System.out.println("Result = " + result);
    }
}
```

```
C:\Users\sylaj\OneDrive\oop program>javac LambdaExplicitMultiStatement.java

C:\Users\sylaj\OneDrive\oop program>java LambdaExplicitMultiStatement
Adding numbers: 10 and 20
Result = 30
```